

Cerberus

Detecção e Bloqueio de Agentes de IA Não Autorizados
na Rede Corporativa

Documentação Técnica

Versão 1.0.0 · 01/10/2026
Classificação: Uso interno — Segurança da Informação

Sumário

1. Visão geral e objetivo
2. Arquitetura da solução
3. Componentes
4. Motor de detecção e assinaturas
5. Varredura de rede e redes segmentadas (VLANs)
6. Relatório consolidado
7. Instalação e implantação
8. Execução em container
9. Configuração
10. Operação, segurança e conformidade
11. Referência de comandos (CLI)
12. Testes e qualidade

1. Visão geral e objetivo

O **Cerberus** é uma ferramenta de conformidade corporativa cujo objetivo é **detectar e bloquear o uso de agentes de IA e túneis de rede não autorizados** nas estações de trabalho e na rede interna, em aderência à política de segurança da organização. São considerados alvos: servidores de LLM locais (Ollama, LM Studio, LocalAI), frameworks de agentes autônomos (CrewAI, AutoGPT, LangGraph), assistentes de código por linha de comando (Claude, Gemini, Aider) e serviços de tunelamento reverso (ngrok, cloudflared, frp, Tailscale).

A solução adota uma abordagem em **camadas complementares**, por ser mais robusta do que qualquer medida isolada: um agente de endpoint que inspeciona processos, uma varredura ativa da rede, geradores de bloqueio de perímetro (DNS e firewall) e um relatório consolidado por equipamento.

Uso autorizado apenas. O Cerberus deve ser implantado somente em equipamentos e redes que a organização administra e tem o direito de auditar. A operação precisa ser aprovada pela área de Segurança/TI e comunicada aos usuários conforme a política interna. O produto inicia sempre em **modo auditoria** (apenas registra); o encerramento de processos é uma ação explícita (opt-in).

2. Arquitetura da solução

O Cerberus é um pacote Python único (comando `cerberus`) que expõe seis subcomandos. Dois mecanismos de detecção coexistem e se complementam:

- **Sentinel (endpoint):** um processo por máquina que inspeciona a árvore de processos local. Independe de topologia de rede e enxerga até agentes que não expõem porta.
- **Varredura (rede):** executada de um ponto central, conecta-se às portas características dos hosts-alvo e confirma o serviço por assinatura HTTP. Cobre a rede sem instalação nas estações, mas só vê agentes com porta aberta.

Ambos produzem **eventos** no mesmo formato (linhas JSON), que o módulo de relatório consolida por equipamento. O bloqueio de perímetro é gerado como artefato de configuração (listas de DNS e regras de firewall) para a equipe de rede aplicar nos equipamentos homologados.

Fluxo de dados

```
[ Estações ] [ Rede interna ]
sentinel --\ cerberus rede --\
\ \
>--> eventos.jsonl <-----/
|
v
cerberus relatorio
|
+-----+-----+
v v v
texto CSV/JSON HTML
```

Camadas de controle

Camada	Componente	Função principal
Endpoint	sentinel	Audita/encerra processos de agentes na máquina local.
Rede	rede / painel	Localiza agentes expostos em faixas de IP.
Perímetro	sinkhole / firewall	Gera bloqueio de DNS e de portas.
Consolidação	relatorio	Inventário por equipamento (agente, IP, máquina, VLAN).

3. Componentes

3.1 Sentinel (cerberus sentinel)

Serviço de monitoramento contínuo baseado na biblioteca `psutil`. A cada ciclo, percorre os processos do sistema e aplica as regras de detecção. Em modo auditoria apenas registra evidências; em modo bloqueio encerra o processo e seus filhos (SIGTERM e, se necessário, SIGKILL). Requer privilégios de root/administrador para enxergar e encerrar processos de outros usuários.

3.2 Varredura de rede (cerberus rede)

Varre faixas CIDR/IP usando um pool de threads. Para cada host, testa as portas-alvo via conexão TCP e, quando abertas, confirma o serviço com uma requisição HTTP ao endpoint de identificação (por exemplo `/api/tags` do Ollama). Resolve o nome da máquina por DNS reverso. Usa apenas a biblioteca padrão do Python.

3.3 Painel desktop (cerberus painel)

Interface gráfica (Tkinter) sobre o motor de varredura. Permite informar as faixas, escolher os serviços, acompanhar o progresso e exportar o resultado em HTML/CSV. Pode ser empacotado como executável standalone (PyInstaller) para distribuição sem Python.

3.4 Bloqueio de perímetro (cerberus sinkhole / firewall)

Geradores de configuração. O `sinkhole` produz listas de bloqueio de DNS nos formatos Pi-hole, hosts, dnsmasq e BIND RPZ. O `firewall` gera regras iptables (Linux) ou netsh (Windows). Nenhum aplica mudanças automaticamente — são artefatos para revisão.

3.5 Relatório (cerberus relatorio)

Consolida um ou mais arquivos de eventos, agrupando por equipamento e juntando múltiplos agentes de um mesmo host. Exporta em texto, JSON, CSV e HTML.

4. Motor de detecção e assinaturas

As regras foram desenhadas para **minimizar falsos positivos**, que num ambiente de produção poderiam interromper trabalho legítimo:

Regra	Critério	Ação
Binário proibido	Nome do processo ou caminho real do executável (detecta agentes renomeados).	Encerrar
Framework em interpretador	Termo na linha de comando, apenas quando o processo é python/node/java. Comandos de instalação (pip/npm) são ignorados.	Encerrar
Porta característica	Escuta em 11434 (Ollama), 1234 (LM Studio), 4040 (ngrok).	Encerrar
Porta genérica	Escuta em 3000/8000/8080 (usadas por apps legítimos).	Apenas alertar
Isenção	Processos de SO e usuários homologados.	Ignorar

As assinaturas padrão cobrem servidores de LLM, frameworks de agentes, assistentes CLI (incluindo Claude/Anthropic e Gemini) e túneis. Podem ser estendidas ou substituídas por um arquivo JSON de política, sem alterar o código.

5. Varredura de rede e redes segmentadas (VLANs)

Em redes com muitas VLANs, um scanner único não alcança os demais segmentos, pois o roteamento/firewall entre sub-redes normalmente bloqueia. O Cerberus oferece duas estratégias, idealmente combinadas:

- **Sentinel por máquina (independe de VLAN):** cada estação se reporta sozinha; é a cobertura mais completa. Distribuído por GPO/Intune (Windows) ou Ansible/SSH (Linux).
- **Varredura por segmento:** um arquivo lista as VLANs; cada detecção é rotulada com o segmento. Quando não há rota até todas as VLANs, roda-se uma instância por segmento, e os eventos são consolidados num único relatório.

A varredura faz apenas **conexões de saída** e não exige abrir porta na máquina que a executa, nem no firewall de host. Em VLANs segmentadas, o firewall entre as sub-redes precisa permitir a saída do host de varredura para as portas dos alvos — ou utiliza-se uma instância por segmento.

6. Relatório consolidado

O relatório agrupa as detecções por equipamento (identificado por host e IP) e lista todos os agentes distintos encontrados em cada um, com a VLAN quando disponível. Máquinas com múltiplos agentes são destacadas. Colunas:

- **segmento (VLAN) · host · IP · agente/modelo**
- **tipo · ação** (auditado/encerrado/detectado-rede) · **ocorrências · usuários · qtd. de agentes na máquina**

Formatos de saída: **texto** (terminal), **JSON** (integração/SIEM), **CSV** (planilha) e **HTML** (apresentação). O HTML destaca visualmente os equipamentos com mais de um agente.

7. Instalação e implantação

7.1 Linux (systemd)

```
sudo ./deploy/instalar-linux.sh # auditoria (recomendado)
sudo ./deploy/instalar-linux.sh --bloquear # encerra os processos
sudo ./deploy/instalar-linux.sh --rede # agenda a varredura (timer)
sudo ./deploy/instalar-linux.sh --desinstalar # remove servicos e timers
```

Cria o serviço cerberus-sentinel, os diretórios /etc/cerberus e /var/log/cerberus, e (com --rede) um service oneshot e um timer que executam a varredura periodicamente, gravando um JSONL com data/hora.

7.2 Windows (Agendador de Tarefas)

```
.\deploy\instalar-windows.ps1 # auditoria (recomendado)
.\deploy\instalar-windows.ps1 -Bloquear # encerra os processos
.\deploy\instalar-windows.ps1 -Desinstalar # remove a tarefa
```

Registra a tarefa Cerberus-Sentinel (inicia no boot como SYSTEM, sem dependências externas) e cria os dados em C:\ProgramData\cerberus.

7.3 Painel como executável standalone

```
./packaging/build-linux.sh # Linux -> dist/cerberus-painel
.\packaging\build-windows.ps1 # Windows -> dist\cerberus-painel.exe
```

8. Execução em container

Por padrão, um container enxerga apenas os próprios processos (namespace de PID isolado). Para o sentinel vigiar a máquina hospedeira, o compose fornecido usa pid: host e network_mode: host:

```
docker compose -f deploy/docker-compose.yml up -d --build
```

A varredura de rede funciona em qualquer container, desde que haja rota de rede até os alvos.

9. Configuração

As assinaturas são estendidas por um JSON de política aplicado com --config. Chaves adicionar_* somam às listas padrão; substituir_* trocam a lista inteira.

```
{
  "adicionar_binarios": ["meuagente-interno"],
  "adicionar_portas_proibidas": { "9100": "Agente proprietario" },
  "adicionar_usuarios_autorizados": ["pesquisa"]
}
```

Para as VLANs, o arquivo de segmentos (ver config/redes.exemplo.json) lista nome e CIDR de cada rede. O painel web auxilia a montar o arquivo de assinaturas marcando os agentes desejados.

10. Operação, segurança e conformidade

- **Inicie em auditoria.** Rode por um período registrando antes de ativar o bloqueio, para medir impacto e refinar isenções.
- **Privilégios.** O sentinel precisa de root/SYSTEM para encerrar processos de outros usuários; sem isso, apenas audita o que consegue ler.
- **Evidências.** Cada evento (host, IP, usuário, regra, comando) é uma linha JSON, pronta para encaminhamento a um SIEM.
- **Exceções.** Usuários/equipes homologados (ex.: laboratório de P&D;) podem ser isentos por configuração.
- **Limitações.** O sentinel detecta agentes conhecidos — mantenha as assinaturas atualizadas e combine com o bloqueio de perímetro.

Aviso legal. O encerramento automático de processos pode interromper o trabalho do usuário. Garanta respaldo na política interna e um canal de exceção para casos homologados. Toda varredura de rede deve ser previamente autorizada.

11. Referência de comandos (CLI)

Comando	Descrição
cerberus sentinel [--bloquear]	Monitora e (opcionalmente) encerra processos locais.
cerberus rede <cidr...> [--arquivo]	Varre faixas de IP; aceita arquivo de segmentos.
cerberus painel	Abre o painel desktop de varredura.
cerberus sinkhole --formato	Gera lista de bloqueio de DNS.
cerberus firewall --plataforma	Gera regras de firewall.
cerberus relatorio <jsonl...>	Consolida os eventos por equipamento.

Todos aceitam --config (assinaturas) e --help. Saídas de relatório/bloqueio aceitam --saida.

12. Testes e qualidade

O projeto acompanha uma suíte de testes automatizados (pytest) que cobre as regras de detecção, isenções, auditoria vs. bloqueio, os geradores de DNS e firewall, a consolidação do relatório, a varredura de rede (com servidor HTTP simulado) e o suporte a segmentos/VLANs. A integração contínua (GitHub Actions) executa a suíte nas versões de Python 3.9 a 3.12.

```
pip install -e ".[dev]"
pytest -q
```

A documentação complementar está no repositório: README.md (guia rápido) e docs/MANUAL.md (manual de instalação e uso).